

PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

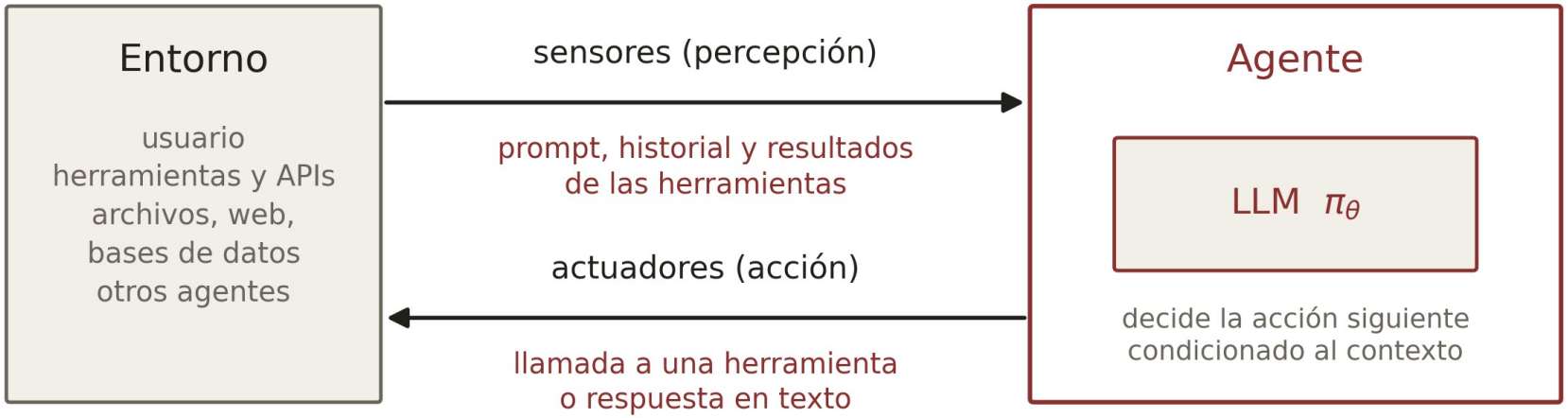
Procesamiento de Lenguaje Natural

github.com/marcelomendoza/IIC3670

Marcelo Mendoza

Agentes

Un agente percibe su entorno a través de sensores, actúa sobre él mediante actuadores y toma decisiones autónomas para alcanzar un objetivo. En un agente LLM, el que decide es el modelo de lenguaje.



en negro, la definición de Russell y Norvig; en rojo, su versión con un LLM

Russell y Norvig, Artificial Intelligence: A Modern Approach (4.ª ed., 2022); Wang et al., Frontiers of Computer Science 2024.

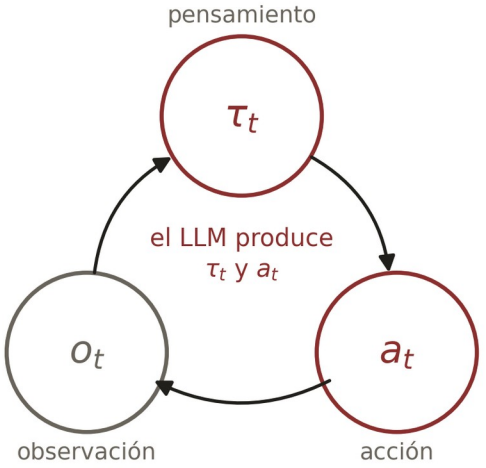
El ciclo ReAct

$$c_t = \big(x,\ \tau_1, a_1, o_1,\ \ldots,\ \tau_{t-1}, a_{t-1}, o_{t-1}\big)$$ contexto en el paso t: la tarea x y la traza

$$\tau_t \sim \pi_\theta(\cdot \mid c_t)$$ pensamiento: texto que no actúa sobre el entorno

$$a_t \sim \pi_\theta(\cdot \mid c_t, \tau_t), \quad a_t \in \mathcal{A} \cup \{\text{finish}[y]\}$$ $\mathcal{A}$: llamadas a herramientas; finish$[y]$ entrega la respuesta y

$$o_t = \text{entorno}(a_t)$$ observación: el resultado se agrega al contexto



x	¿cuántos habitantes tiene la capital de Ñuble?
τ_1	Necesito saber cuál es la capital de Ñuble.
a_1	`search[Región de Ñuble]`
o_1	… la capital regional es Chillán …
τ_2	Ahora busco la población de Chillán.
a_2	`search[Chillán]`
o_2	… 184.739 habitantes (censo 2017) …
τ_3	Tengo la respuesta.
a_3	`finish[184.739 habitantes]`

traza ilustrativa

Yao et al., ICLR 2023 (ReAct: Synergizing Reasoning and Acting in Language Models).

Workflow y agente

No todo sistema con un LLM es un agente. La diferencia está en quién decide el paso siguiente.

Workflow

el código fija el flujo

Los pasos y su orden están escritos en el programa.

El LLM resuelve cada paso, pero no decide cuál sigue.

Es predecible y su costo es conocido.

Ejemplos: encadenar prompts, rutear por clase, paralelizar.

Agente

el LLM decide el flujo

El modelo elige la acción siguiente según lo que observa.

El número de pasos depende de la tarea.

Termina con una respuesta final o un máximo de pasos.

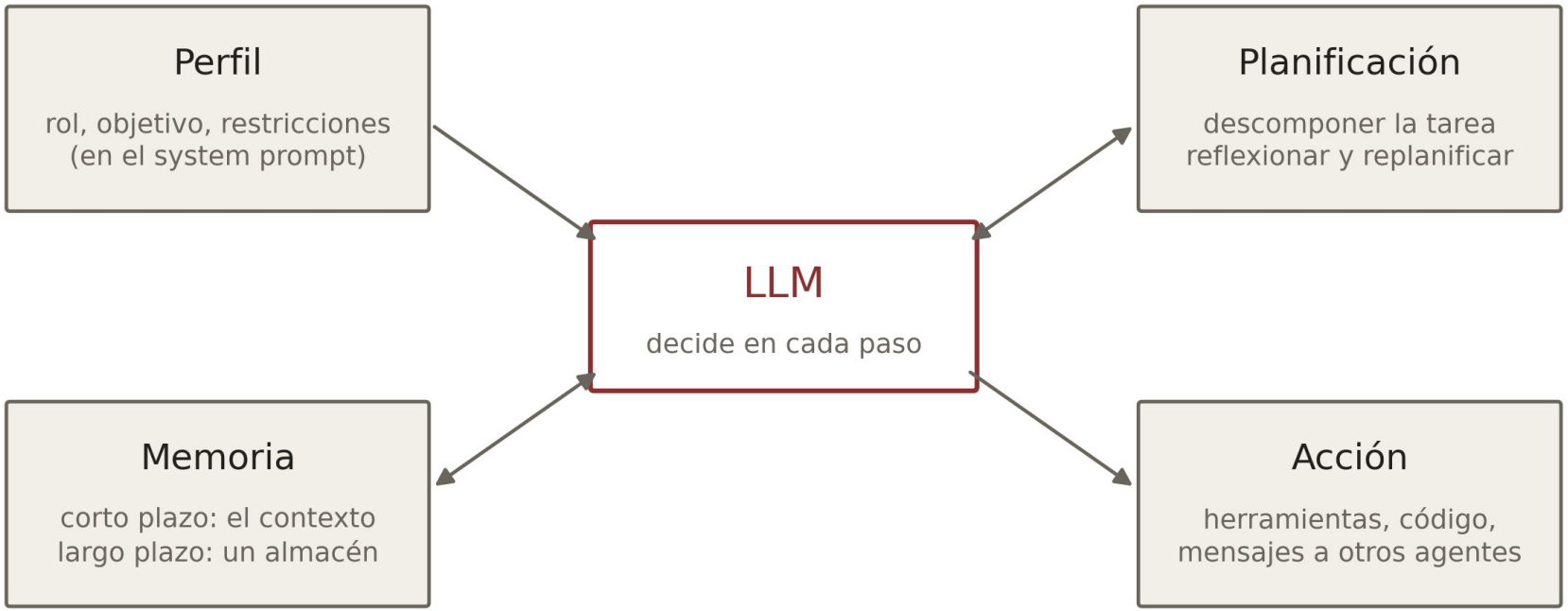
Conviene cuando los pasos no se conocen de antemano.

Fuente

Anthropic, Building Effective Agents (2024).

Componentes de un agente

La arquitectura habitual de un agente LLM tiene cuatro módulos alrededor del modelo: perfil, memoria, planificación y acción.



Wang et al., Frontiers of Computer Science 2024 (A Survey on Large Language Model based Autonomous Agents).

Memoria de largo plazo

Lo que no cabe en el contexto se almacena como una lista de recuerdos en texto. En cada paso se recuperan los que obtienen mayor puntaje para la consulta actual.

$$\text{puntaje}(m) = \alpha_{\text{rec}}\,\mathbf{rec}(m) + \alpha_{\text{imp}}\,\mathbf{imp}(m) + \alpha_{\text{rel}}\,\mathbf{cos}\,(e_q, e_m)$$

cada término normalizado a [0,1], con los tres α=1

$$\text{rec}(m) = 0{,}995^{\,\Delta h_m}$$

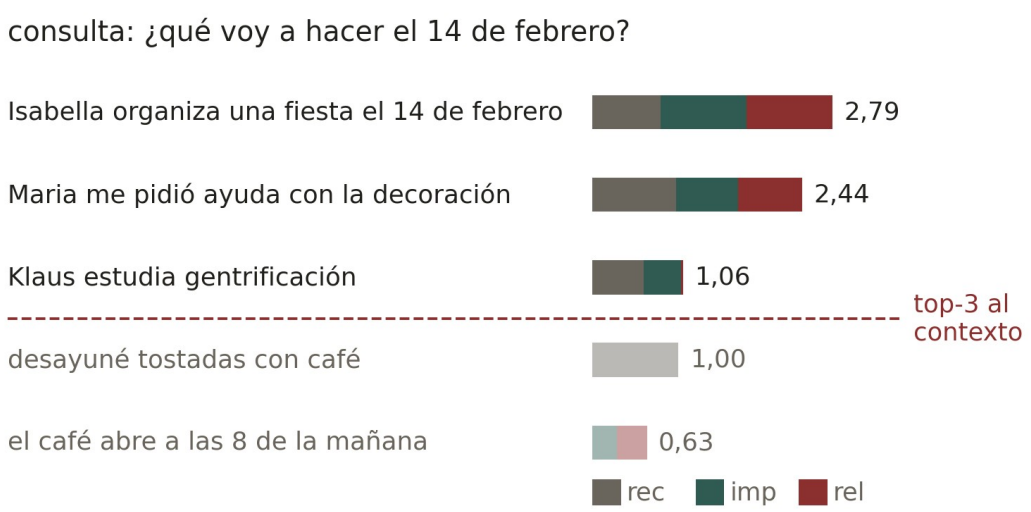
Δh_m: horas de simulación desde el último acceso a m

$$\text{imp}(m) \in \{1, \ldots, 10\}$$

la asigna el LLM al crear el recuerdo

$$e_q,\ e_m$$

embeddings de la consulta y del recuerdo



Recencia

Decae en forma exponencial con las horas transcurridas desde el último acceso al recuerdo.

Importancia

El LLM la califica de 1 a 10 al crear el recuerdo. Un desayuno vale poco y una fiesta, mucho.

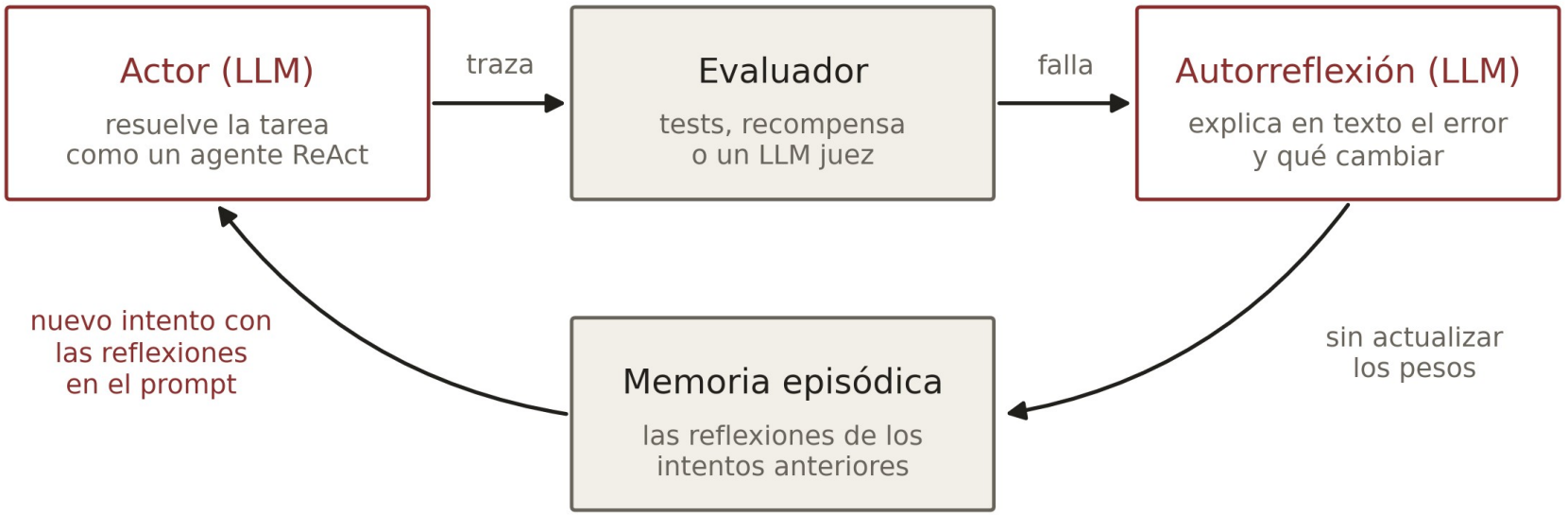
Relevancia

El coseno entre los embeddings de la consulta y del recuerdo.

Park et al., UIST 2023 (Generative Agents). Ejemplo ilustrativo con la fórmula del paper.

Reflexión

Cuando un intento falla, el agente escribe en texto qué salió mal y lo guarda en memoria. El intento siguiente lleva esas reflexiones en el prompt.



Shinn et al., NeurIPS 2023 (Reflexion: Language Agents with Verbal Reinforcement Learning).

Retrieval-Augmented Generation

El conocimiento de un LLM está en sus parámetros, fijado al terminar el entrenamiento. RAG recupera pasajes de una colección externa y condiciona la generación en ellos.

$$p(y \mid x) \approx \sum_{z \in \text{top-}k} p_\eta(z \mid x) \prod_{t=1}^{|y|} p_\theta(y_t \mid x, z, y_{<t})$$

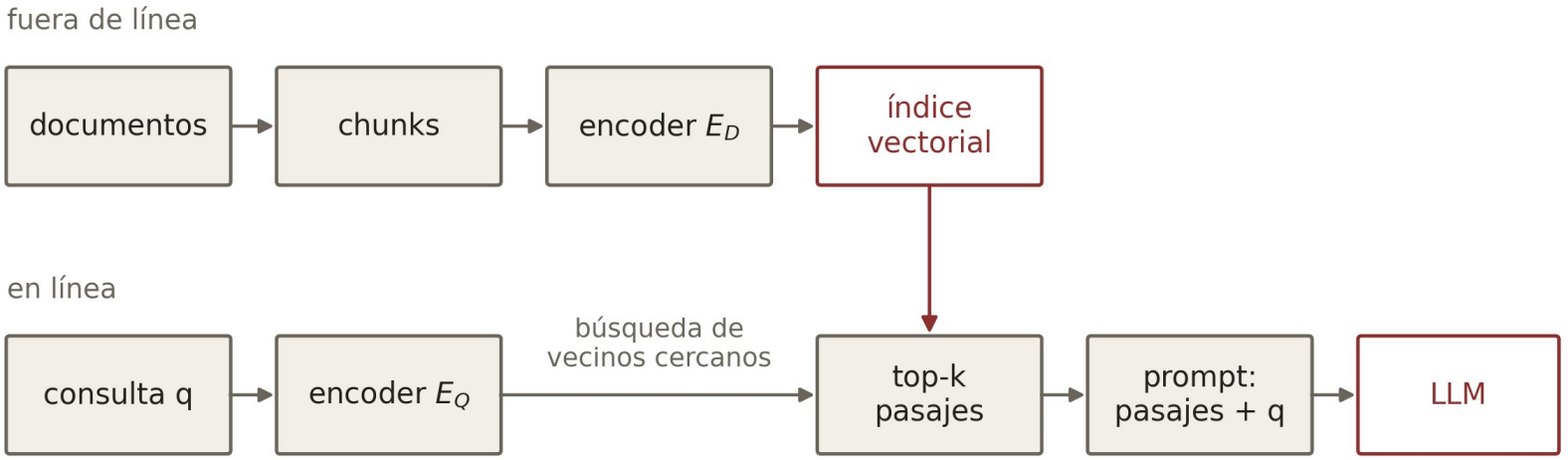
se marginaliza sobre los k pasajes recuperados

$$p_\eta(z \mid x) \propto \exp\big(E_D(z)^\top E_Q(x)\big)$$

retriever: dos encoders, consulta y pasaje

p_θ

generador: el modelo de lenguaje condicionado a x y z



Lewis et al., NeurIPS 2020 (Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks).

El retriever

El retriever asigna un puntaje a cada par consulta y pasaje. Las cuatro funciones difieren en cuánto se puede calcular antes de conocer la consulta.

$$s_{\text{BM25}}(q, d_j) = \sum_{t_i \in q} w_{i,j} \qquad w_{i,j}\text{: peso BM25 del término } t_i \text{ en } d_j$$

$$s_{\text{cross}}(q, d) = v^\top \text{BERT}\big([\text{CLS}]\, q\, [\text{SEP}]\, d\big)_{[\text{CLS}]} \qquad \text{un forward por cada par}$$

$$\boldsymbol{s_{\text{bi}}(q, d) = E_Q(q)^\top E_D(d)} \qquad E_D(d) \text{ se indexa una vez}$$

$$s_{\text{ColBERT}}(q, d) = \sum_{i \in q} \max_{j \in d} \mathbf{q}_i^\top \mathbf{d}_j \qquad \text{un embedding por token}$$

Cross-encoder

El más preciso, pero exige un forward por pasaje. Se usa para reordenar pocos candidatos.

Bi-encoder

DPR (Dense Passage Retrieval). Los embeddings de los pasajes se precalculan y la búsqueda es un producto punto.

ColBERT

Un embedding por token del pasaje, con la máxima similitud por token de la consulta.

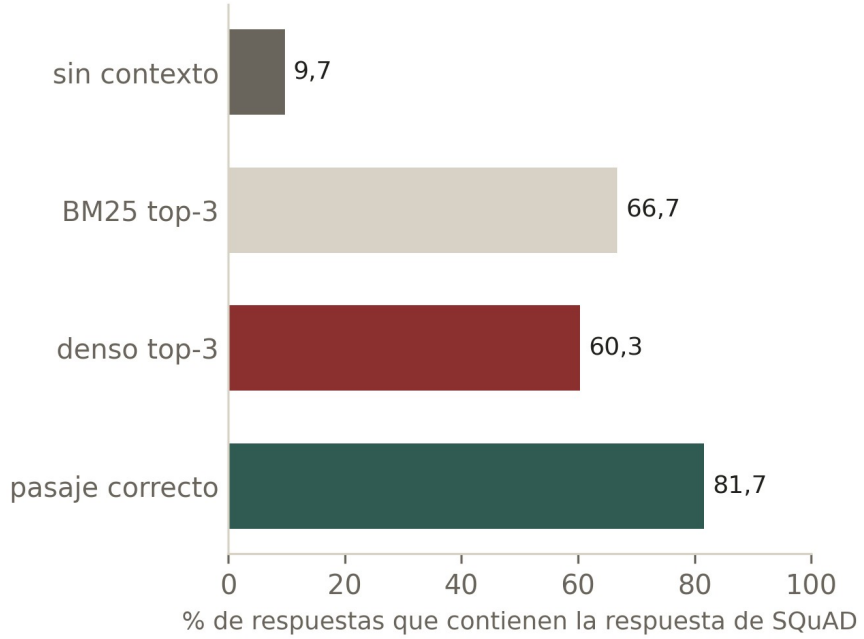
Karpukhin et al., EMNLP 2020 (DPR); Khattab y Zaharia, SIGIR 2020 (ColBERT).

Generar con los pasajes

Incrustar los pasajes en el prompt equivale a hacer completación de texto condicionada a ellos. Qwen2.5-1.5B-Instruct con 300 preguntas de SQuAD, recuperando de sus 20.958 párrafos.

$$P\big(w_t \mid \text{Q:},\ q,\ \text{A:},\ w_{<t}\big)$$ sin RAG: la respuesta sale de los parámetros

$$P\big(w_t \mid \boldsymbol{R(q)},\ \text{Q:},\ q,\ \text{A:},\ w_{<t}\big)$$ $\boldsymbol{R(q)}$: los k pasajes recuperados con q



Sin contexto

El modelo acierta en el 10 % de las preguntas.

Con RAG

Con los tres pasajes recuperados sube a 60 % con un bi-encoder (denso) y 67 % con BM25.

El límite

Con el pasaje correcto llega a 82 %. La diferencia restante es error de recuperación.

Lewis et al., NeurIPS 2020. Cálculo propio con Qwen2.5-1.5B-Instruct.

Agentic RAG

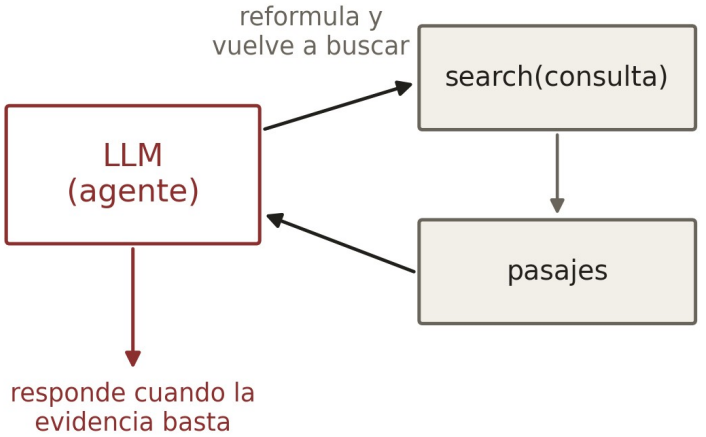
En un agente, la recuperación es una herramienta más. El modelo decide si buscar, con qué consulta y cuántas veces, y responde cuando la evidencia basta.

RAG fijo, con una sola búsqueda



la búsqueda ocurre aunque no haga falta, y una sola vez

agentic RAG, donde el agente decide



Cuatro criterios para dejar de buscar

El propio modelo

Lee los pasajes, juzga si tiene lo necesario y emite finish.

ReAct

Decisión aprendida

Un crítico inserta tokens de «recuperar» y «respaldado» en textos de ajuste, y el modelo aprende a emitirlos.

Self-RAG

Incertidumbre

Si la oración siguiente tiene tokens de baja probabilidad, la usa como consulta.

FLARE

Juez externo

Un evaluador clasifica los pasajes como correctos, ambiguos o incorrectos.

CRAG

Yao et al., ICLR 2023 (ReAct); Asai et al., ICLR 2024 (Self-RAG); Jiang et al., EMNLP 2023 (FLARE); Yan et al., arXiv 2024 (CRAG).

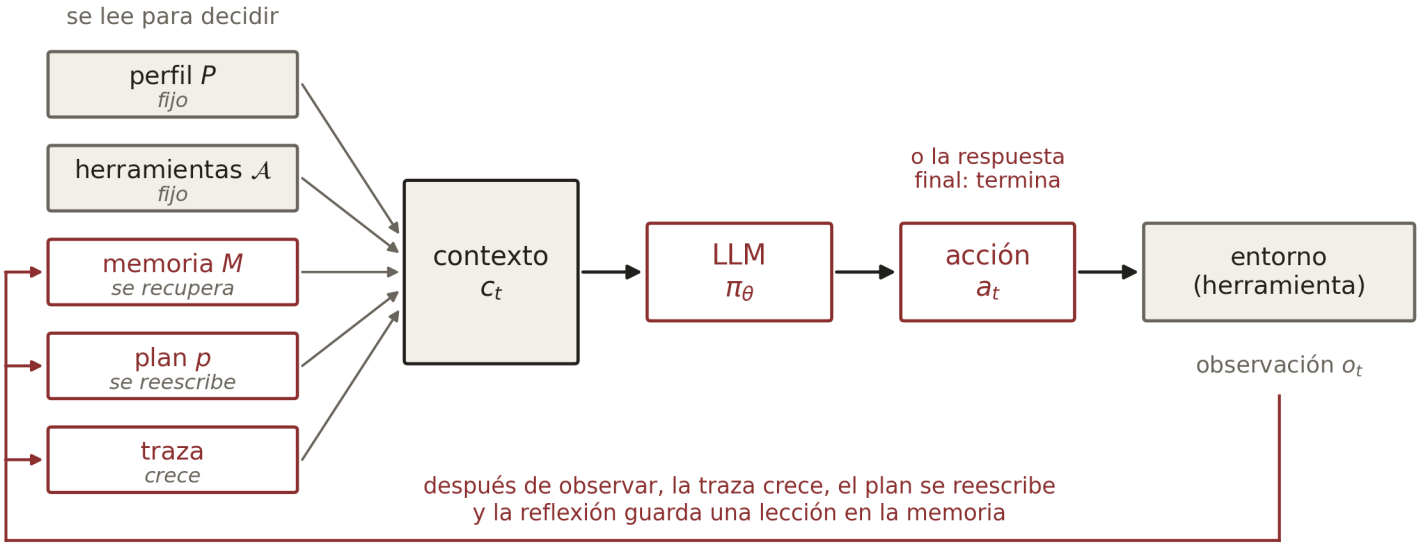
Cómo decide un agente

Los pesos θ no cambian durante la tarea. Perfil, memoria, reflexión y plan influyen en la decisión solo a través de lo que ponen en el contexto.

$$c_t = \big(P,\ \mathcal{A},\ \boldsymbol{M_t},\ \boldsymbol{p_t},\ x,\ \tau_1, a_1, o_1, \ldots, o_{t-1}\big) \qquad \text{lo que el agente ve en el paso } t$$

$$\boldsymbol{M_t} = \text{los } k \text{ recuerdos de mayor puntaje para el paso } t \qquad \text{incluye las reflexiones guardadas}$$

$$a_t \sim \pi_\theta(\cdot \mid c_t), \qquad \theta \text{ fijo} \qquad \text{una llamada en } \mathcal{A} \text{ o la respuesta final}$$



Wang et al., Frontiers of Computer Science 2024; Yao et al., ICLR 2023 (ReAct); Shinn et al., NeurIPS 2023 (Reflexion).

Una decisión, paso a paso

Cada capa del contexto explica una parte de la acción elegida. Si se quita una capa, la acción cambia.



Ejemplo ilustrativo, con el ciclo de ReAct (Yao et al., ICLR 2023) y la memoria de Reflexion (Shinn et al., NeurIPS 2023).

¿Hasta qué punto es autónomo?

La autonomía es una decisión de diseño, independiente de la capacidad del modelo. Se define por el papel que conserva el usuario.

Nivel	El usuario	El agente
1. operador	dirige	ayuda a pedido
2. colaborador	planifica con el agente	ejecuta con él
3. consultor	aporta criterio	planifica y ejecuta
4. aprobador	aprueba lo riesgoso	hace el resto solo
5. observador	solo puede detenerlo	trabaja solo

Lo que acota la autonomía

El agente solo puede llamar las herramientas declaradas.

Un presupuesto de pasos o de tokens corta el ciclo.

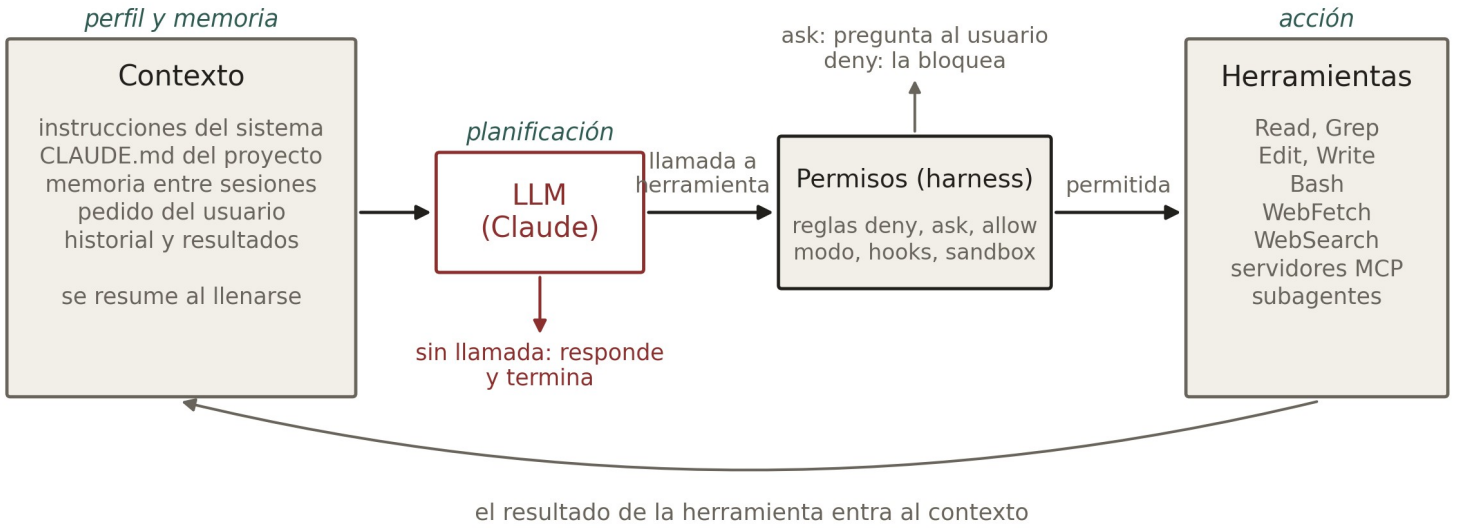
Las acciones de riesgo esperan la aprobación de una persona.

Las reglas de permisos las aplica el programa que rodea al modelo, no el modelo.

Feng, McDonald y Zhang, Knight First Amendment Institute 2025 (Levels of Autonomy for AI Agents).

Un agente real: Claude Code

Un agente de programación que trabaja en la terminal. El LLM decide la acción siguiente, y el programa que lo rodea (el harness) decide si se ejecuta.



Niveles de autonomía, de menor a mayor

Plan

Lee el código y propone un plan, sin editar archivos.

Manual

Pide aprobación antes de editar archivos o ejecutar comandos.

Accept edits

Edita sin preguntar, pero pide aprobación para otros comandos.

Auto

Un clasificador revisa las acciones y bloquea las riesgosas.

Anthropic, documentación de Claude Code (2026).

Sistemas multi-agente

Un sistema multi-agente reparte la tarea entre varios agentes LLM que se comunican. El costo es más llamadas al modelo y la necesidad de coordinarlas.

Wu et al., COLM 2024 (AutoGen); Anthropic, blog de ingeniería, 2025; Cemri et al., NeurIPS 2025 (MAST).

Especialización

Cada agente tiene su perfil, sus herramientas y un prompt corto enfocado en una parte de la tarea.

un rol por agente

Contexto

Cada agente trabaja con su propia ventana de contexto. Ninguno necesita la traza completa del sistema.

contexto más corto

Paralelismo

Las subtareas independientes corren al mismo tiempo, en agentes distintos.

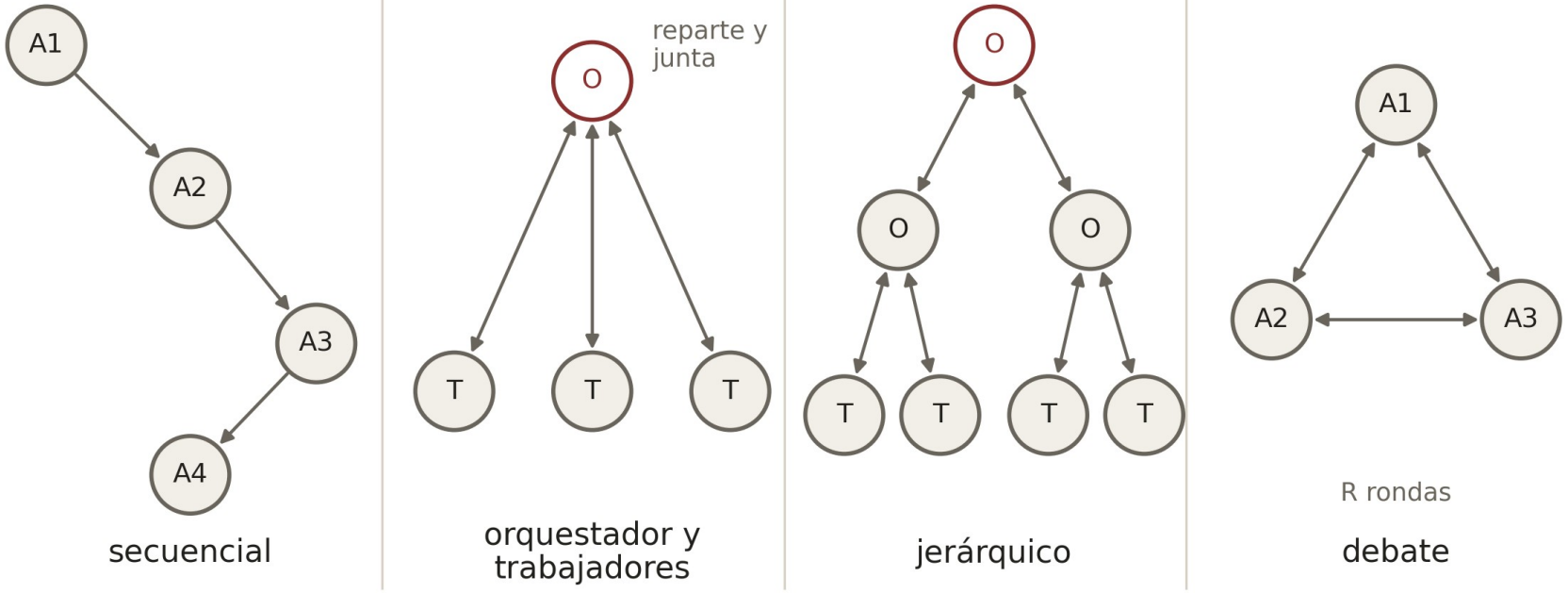
menor latencia

El gran desafío es la coordinación

En más de 1.600 trazas de 7 frameworks, las fallas vienen del diseño del sistema (44 %), del desalineamiento entre agentes (32 %) y de la verificación del resultado (24 %), y no solo de la capacidad del modelo.

Esquemas de interacción

La organización de los agentes define quién habla con quién y quién decide. O es un orquestador, T un trabajador y A un agente par.

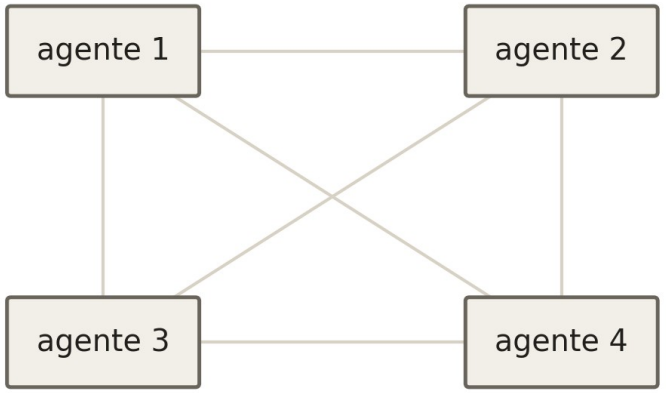


Anthropic, Building Effective Agents (2024); Du et al., ICML 2024; Hong et al., ICLR 2024 (MetaGPT).

Comunicación entre agentes

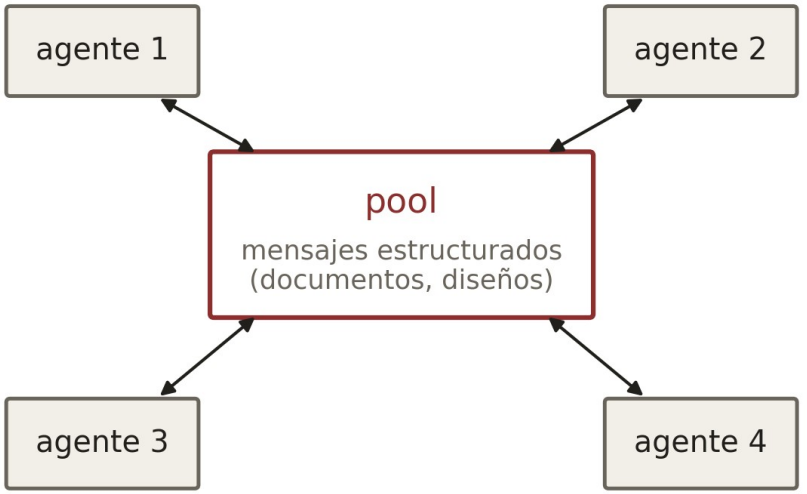
Los agentes se comunican por mensajes directos o a través de un espacio compartido donde publican sus resultados y del que leen lo que les corresponde.

paso de mensajes



cada par mantiene su conversación

pool de mensajes compartido



publica, y lee solo lo que su rol suscribe

Hong et al., ICLR 2024 (MetaGPT); Li et al., NeurIPS 2023 (CAMEL).

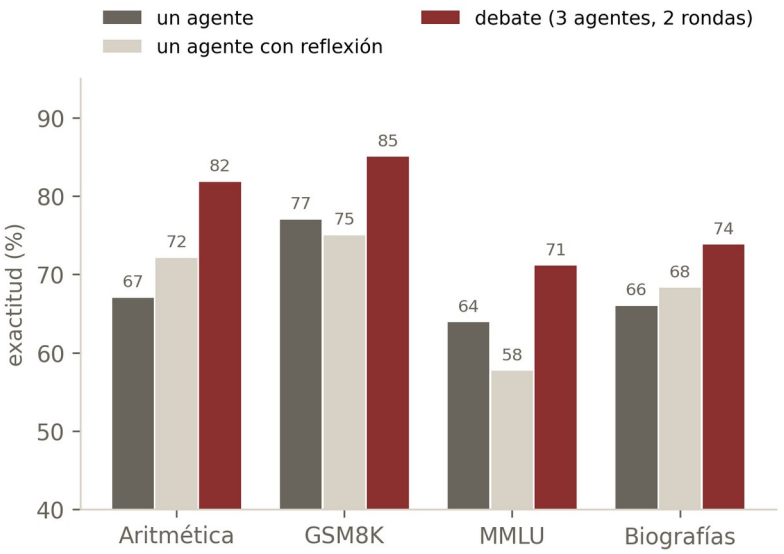
Debate entre agentes

Varios agentes responden la misma pregunta y, en cada ronda, revisan su respuesta después de leer las de los demás.

$$y_i^{(0)} \sim \pi_\theta(\cdot \mid x) \qquad i{=}1,\dots,N\text{: cada agente responde por separado}$$

$$y_i^{(r)} \sim \pi_\theta\big(\cdot \mid x,\ y_i^{(r-1)},\ \{y_j^{(r-1)}\}_{j\neq i}\big) \qquad r{=}1,\dots,R\text{: lee las respuestas de los demás y actualiza la suya}$$

$$\hat{y} = \text{mayoría}\big(y_1^{(R)},\dots,y_N^{(R)}\big) \qquad \text{la respuesta final}$$



Contra un agente

El debate mejora en las cuatro tareas, con 3 agentes y 2 rondas.

Contra la reflexión

Revisar la propia respuesta no basta. En GSM8K y MMLU la reflexión empeora.

Las tareas

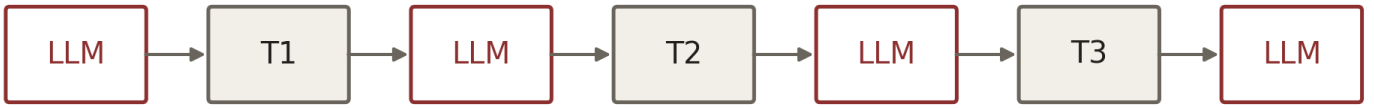
GSM8K son problemas de matemática escolar y MMLU, preguntas de selección múltiple de 57 materias.

Du et al., ICML 2024 (Improving Factuality and Reasoning in Language Models through Multiagent Debate), con gpt-3.5-turbo.

Planificar antes o durante

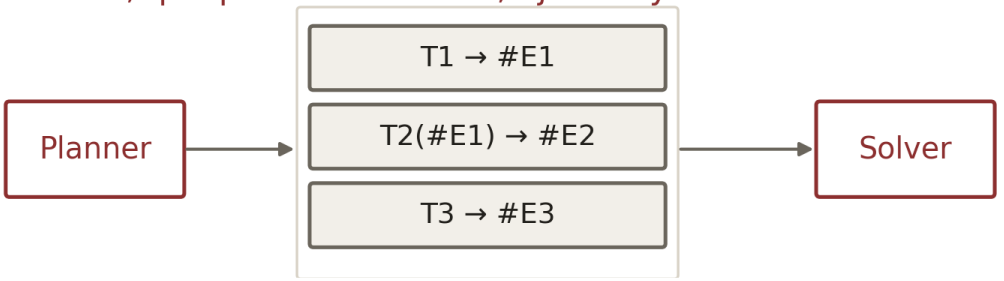
El planner es el componente que descompone la tarea en pasos, cada uno con su herramienta y sus argumentos. ReAct planifica un paso a la vez, y ReWOO escribe el plan completo antes de ejecutar.

ReAct, que vuelve a llamar al LLM con todo el contexto después de cada herramienta



4 llamadas al LLM, contexto creciente

ReWOO, que planifica una vez, ejecuta y resuelve



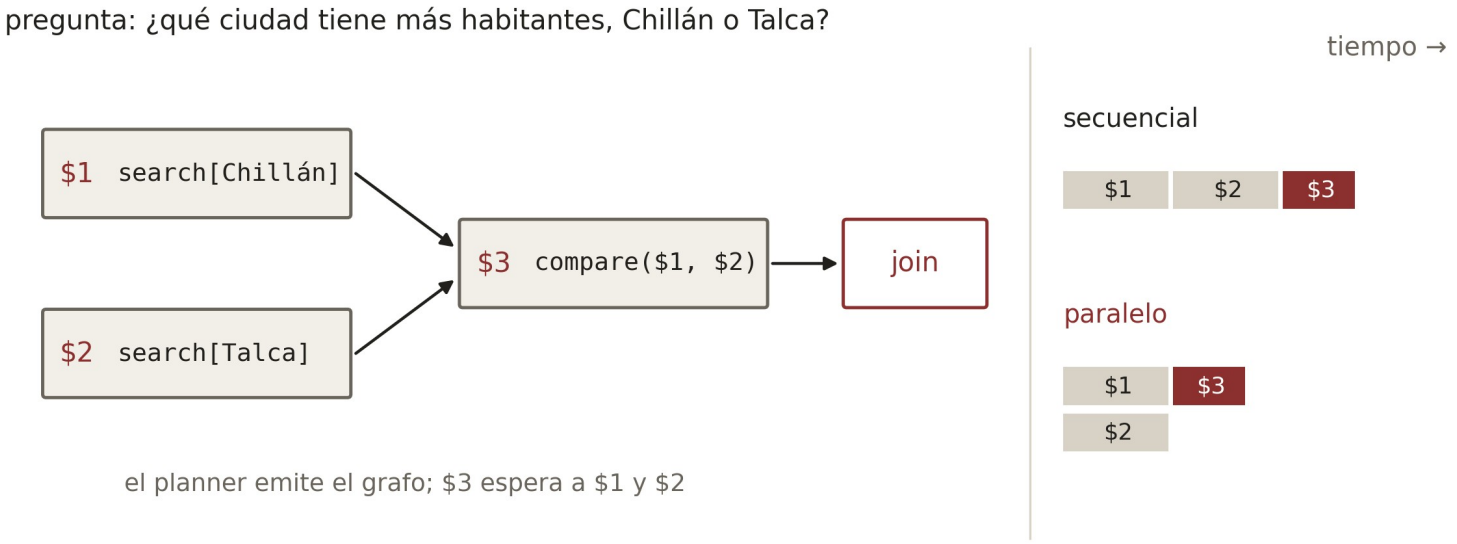
el worker ejecuta el plan

2 llamadas al LLM; las variables #E se llenan con los resultados

ReWOO viene de Reasoning WithOut Observation, porque el planner escribe el plan sin ver los resultados de las herramientas. Yao et al., ICLR 2023 (ReAct); Xu et al., arXiv 2023 (ReWOO); Wang et al., ACL 2023 (Plan-and-Solve).

El plan como grafo de tareas

Si el plan declara qué tarea depende de cuál, las independientes se ejecutan en paralelo. Es el scheduling de un compilador aplicado a llamadas a herramientas.



El paralelismo en tres frameworks

LangChain

Arma un agente con el ciclo ReAct. Si el modelo pide varias herramientas en un mismo paso, las ejecuta en paralelo.

agente listo, construido sobre LangGraph

LangGraph

El programador define un grafo de nodos con estado compartido. Los nodos sin dependencia corren en paralelo.

grafo explícito, con checkpoints

CrewAI

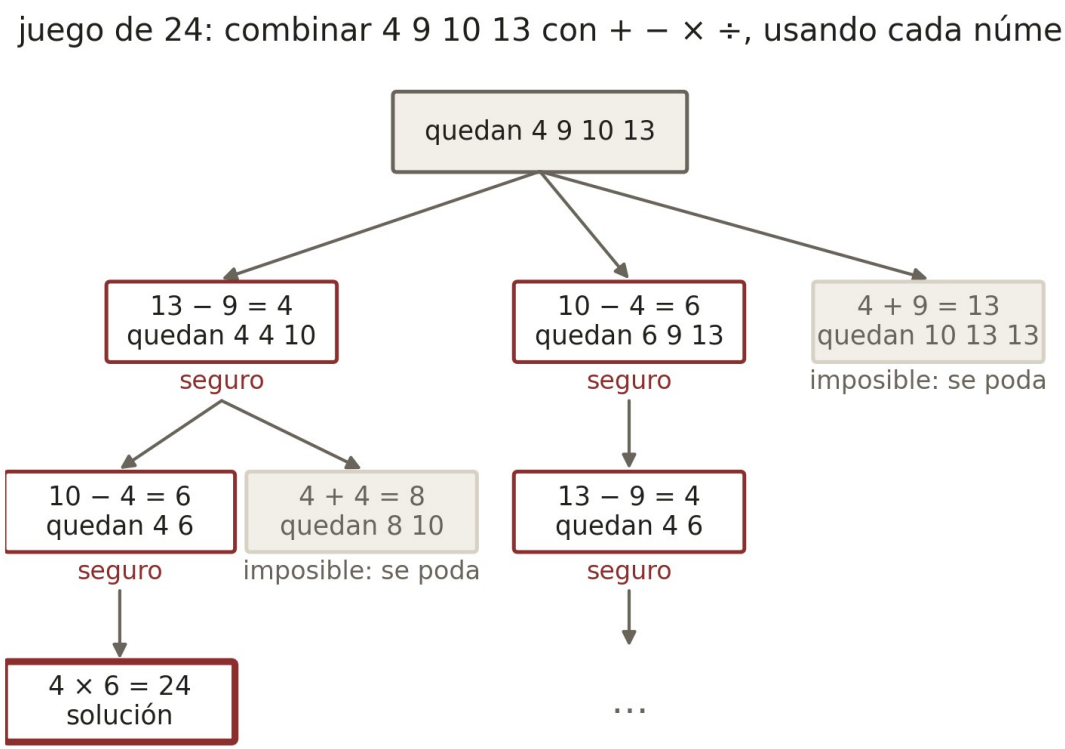
Agentes con rol y tareas, en orden secuencial o con un gerente que delega. Una tarea asíncrona no bloquea a las siguientes.

roles y equipos

Kim et al., ICML 2024 (An LLM Compiler for Parallel Function Calling). Documentación de LangChain, LangGraph y CrewAI (2026).

Buscar sobre planes

Tree of Thoughts trata el problema como un árbol. Desde cada estado el LLM propone varios pasos, evalúa a dónde llevan y una búsqueda expande solo los prometedores.



paso 1

paso 2

paso 3

estado
los números que quedan

pensamiento
una operación entre dos números; el LLM propone varias

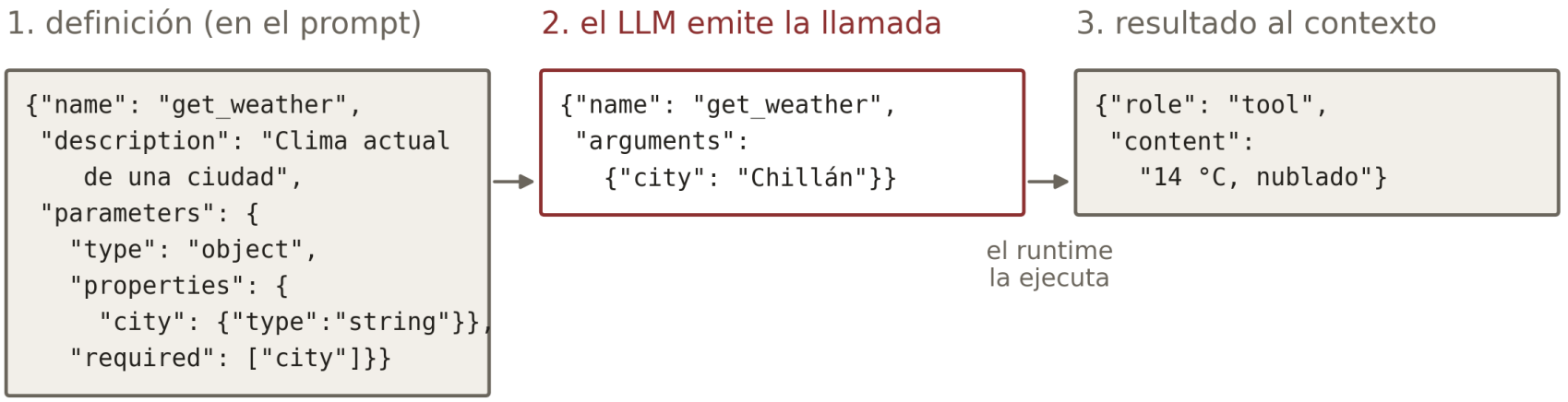
evaluación
el LLM juzga si desde ese estado aún se llega a 24: seguro, probable o imposible

búsqueda
en anchura, conserva los b mejores estados de cada paso

Yao et al., NeurIPS 2023 (Tree of Thoughts); Zhou et al., ICML 2024 (LATS).

Function calling

Cada herramienta se describe con un nombre, una descripción y un esquema JSON de sus argumentos. El modelo, ajustado para eso, emite la llamada con ese formato.

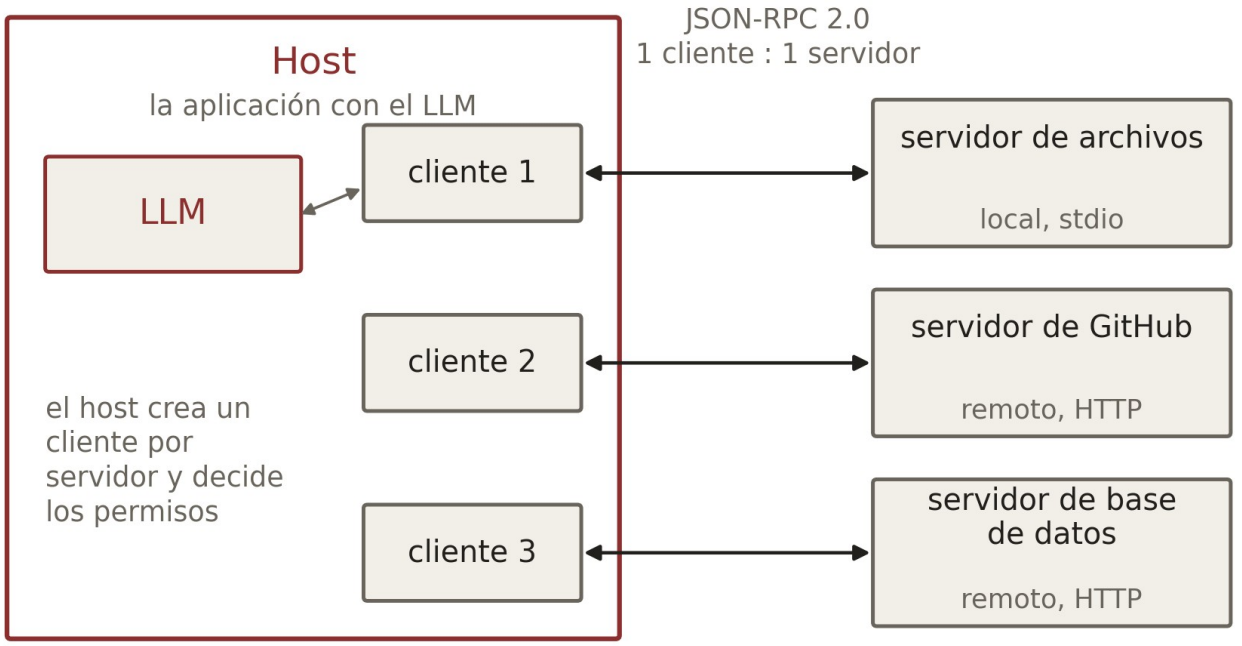


el LLM no ejecuta nada: produce texto con la forma del esquema, y el programa que lo rodea valida los argumentos, llama a la función y devuelve el resultado como un mensaje más

Schick et al., NeurIPS 2023 (Toolformer); JSON Schema.

Model Context Protocol

MCP es un protocolo abierto para conectar aplicaciones con LLM a fuentes de datos y herramientas. Una herramienta se implementa una vez como servidor y la usa cualquier aplicación compatible.



lo que ofrece un servidor

`tools`

funciones que el modelo decide invocar

`resources`

datos de contexto (archivos, registros)

`prompts`

plantillas que elige el usuario

https://modelcontextprotocol.io/specification

Anthropic, Model Context Protocol (2024), especificación en modelcontextprotocol.io.

Otras formas de actuar

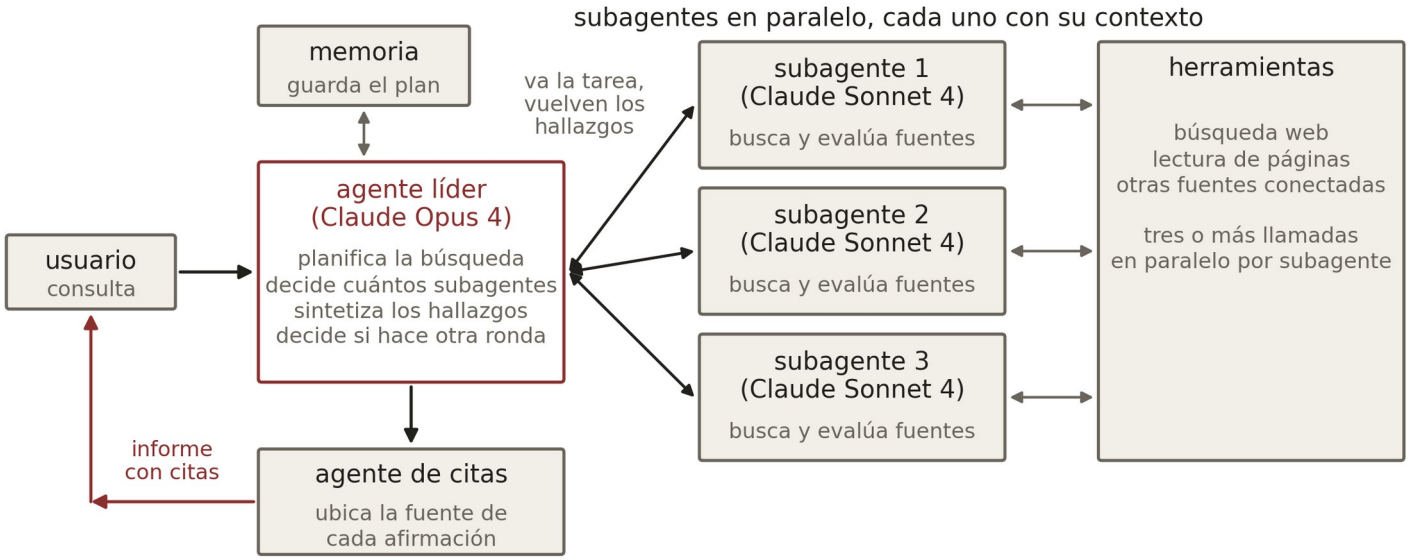
Además de las llamadas en JSON, un agente puede actuar escribiendo código, operando una interfaz gráfica o delegando en otro agente.

Vía	La acción es	Para qué sirve
function calling y MCP	un JSON con nombre y argumentos	llamar herramientas definidas de antemano
código como acción (CodeAct)	un programa en Python que se ejecuta	componer herramientas, iterar y calcular en una sola acción
uso del computador	un clic, una tecla o una captura de pantalla	operar interfaces gráficas sin API
agente a agente (A2A)	un mensaje con una tarea para otro agente	delegar en agentes de otros proveedores

Wang et al., ICML 2024 (CodeAct: Executable Code Actions Elicit Better LLM Agents); Google, Agent2Agent Protocol (2025).

Un sistema multi-agente real

El sistema de investigación de Anthropic sigue el esquema orquestador y trabajadores. Un agente líder reparte la búsqueda entre subagentes que trabajan en paralelo.



Lo que mostró en producción

Calidad

Supera en 90,2 % a un solo agente Opus 4 en su evaluación interna.

Costo

Usa unas 15 veces los tokens de un chat, contra unas 4 de un solo agente.

Tiempo

El paralelismo reduce hasta en 90 % el tiempo de las consultas complejas.

Fallas

Subagentes de más para consultas simples y trabajo duplicado si la delegación es vaga.

Anthropic, How We Built Our Multi-Agent Research System (blog de ingeniería, 2025).